

EXPERIMENT-3

Policy and value function representation using bellman Equations

Name: AKHILA POCHIMI REDDY

Reg no: 192425225

Course code: MLA0305

Course Name: REINFORCEMENT LEARNING

Aim

To implement **policy evaluation using the Bellman equation** and represent the **policy and value function** for a Grid World environment using Python.

Procedure

1. Create a **4 × 4 Grid World** with a starting state and a terminal goal state.
2. Define a fixed policy with actions **Up, Down, Left, and Right**.
3. Initialize the value function of all states to zero.
4. Apply the **Bellman equation** to calculate the value of each state.
5. Repeat the value calculation until the values converge.
6. Generate a **summary table** containing the state, policy action, and value function.
7. Visualize the Grid World using arrows for the policy and values for each state.
8. Display the path from the **START state to the GOAL state**.

Python code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle, Circle
from IPython.display import display

# =====
# EXPERIMENT 3
# Policy and Value Function using Bellman Equation
```

```
# 3 x 3 Grid World
```

```
# =====
```

```
rows, cols = 3, 3
```

```
start = (0, 0)
```

```
goal = (2, 2)
```

```
gamma = 0.9
```

```
reward = -1
```

```
theta = 0.001
```

```
# Fixed policy
```

```
policy = {
```

```
    (0, 0): "R",
```

```
    (0, 1): "R",
```

```
    (0, 2): "D",
```

```
    (1, 0): "R",
```

```
    (1, 1): "R",
```

```
    (1, 2): "D",
```

```
    (2, 0): "R",
```

```
    (2, 1): "R"
```

```
}
```

```
moves = {
```

```
    "U": (-1, 0),
```

```
    "D": (1, 0),
```

```
    "L": (0, -1),
```

```
    "R": (0, 1)
```

```
}
```

```
# =====
```

```
# BELLMAN POLICY EVALUATION
```

```
# =====
```

```
V = np.zeros((rows, cols))
```

```
iteration = 0
```

```

while True:
    delta = 0
    for i in range(rows):
        for j in range(cols):
            if (i, j) == goal:
                continue
            action = policy[(i, j)]
            di, dj = moves[action]
            ni = i + di
            nj = j + dj
            # Boundary condition
            if ni < 0 or ni >= rows:
                ni = i
            if nj < 0 or nj >= cols:
                nj = j
            # Bellman equation
            new_value = reward + gamma * V[ni, nj]
            delta = max(
                delta,
                abs(new_value - V[i, j])
            )
            V[i, j] = new_value
        iteration += 1
    if delta < theta:
        break

# =====
# SUMMARY TABLE
# =====

data = []

for i in range(rows):

```

```

for j in range(cols):
    state_no = i * cols + j + 1
    if (i, j) == goal:
        action = "GOAL"
        value = 0.00
    else:
        action = policy[(i, j)]
        value = round(V[i, j], 3)
    data.append([
        f"S{state_no}",
        action,
        value
    ])
summary = pd.DataFrame(
    data,
    columns=["State", "Policy", "Value"]
)
print("\n===== SUMMARY TABLE =====\n")
display(summary)
print("Discount Factor ( $\gamma$ ):", gamma)
print("Reward:", reward)
print("Iterations:", iteration)
# =====
# VISUALIZATION
# =====
fig, ax = plt.subplots(figsize=(7, 7))
ax.set_xlim(-0.5, 2.5)
ax.set_ylim(2.5, -0.5)
ax.set_aspect("equal")
# Draw grid

```

```

for i in range(3):
    for j in range(3):
        cell = Rectangle(
            (j - 0.5, i - 0.5),
            1,
            1,
            fill=False,
            linewidth=2
        )
        ax.add_patch(cell)

# =====
# OPTIMAL PATH
# START → → ↓ ↓ → GOAL
# =====

path = [
    (0, 0),
    (0, 1),
    (0, 2),
    (1, 2),
    (2, 2)
]

path_x = [p[1] for p in path]
path_y = [p[0] for p in path]
ax.plot(
    path_x,
    path_y,
    linestyle="--",
    linewidth=3,
    marker="o",
    markersize=7

```

```

)
# =====
# START
# =====

start_circle = Circle(
    (0, 0),
    0.17,
    fill=False,
    linewidth=3
)
ax.add_patch(start_circle)
ax.text(
    0,
    -0.28,
    "START",
    ha="center",
    fontsize=10,
    fontweight="bold"
)
# =====
# GOAL
# =====

goal_box = Rectangle(
    (1.70, 1.70),
    0.60,
    0.60,
    fill=False,
    linewidth=3
)
ax.add_patch(goal_box)

```

```

ax.text(
    2,
    2,
    "GOAL",
    ha="center",
    va="center",
    fontsize=10,
    fontweight="bold"
)

# =====
# POLICY ARROWS + VALUES
# =====

arrows = {
    "U": "↑",
    "D": "↓",
    "L": "←",
    "R": "→"
}

for i in range(3):
    for j in range(3):
        if (i, j) == goal:
            ax.text(
                j,
                i + 0.28,
                "V = 0.00",
                ha="center",
                fontsize=9
            )
        else:
            # Policy

```

```

    ax.text(
        j,
        i - 0.08,
        arrows[policy[(i, j)]],
        ha="center",
        fontsize=24,
        fontweight="bold"
    )
# Value
ax.text(
    j,
    i + 0.27,
    f"V = {V[i,j]:.2f}",
    ha="center",
    fontsize=9
)

# =====
# TITLE
# =====

ax.set_title(
    "3×3 Grid World: Policy and Value Function",
    fontsize=15,
    fontweight="bold",
    pad=18
)
ax.text(
    1,
    3.05,
    "START → Optimal Policy → GOAL",
    ha="center",

```



```

    fontsize=10
)
ax.set_xticks([])
ax.set_yticks([])
plt.tight_layout()
plt.show()

# =====

# RESULT

# =====

print("\nRESULT:")
print(
    "The policy and value function were successfully evaluated "
    "using the Bellman equation, and the optimal path from "
    "START to GOAL was clearly visualized."
)

```

Output:

Summary Table

===== SUMMARY TABLE =====

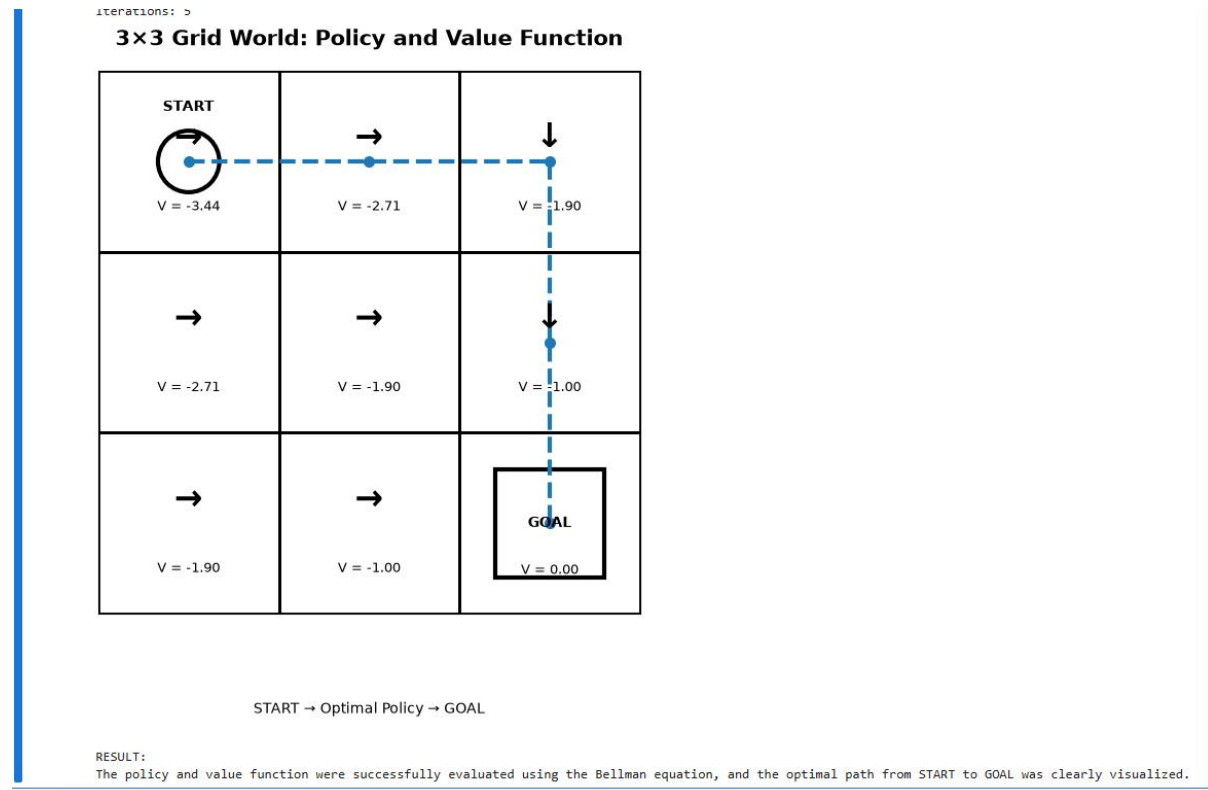
	State	Policy	Value
0	S1	R	-3.439
1	S2	R	-2.710
2	S3	D	-1.900
3	S4	R	-2.710
4	S5	R	-1.900
5	S6	D	-1.000
6	S7	R	-1.900
7	S8	R	-1.000
8	S9	GOAL	0.000

Discount Factor (γ): 0.9

Reward: -1

Iterations: 5

Visualization:



Result

The **policy and value function** were successfully represented using the **Bellman equation**. The value of each state was calculated through iterative policy evaluation, and the path from the starting state to the goal was clearly visualized.